How does asyncio actually work?

Asked 5 years, 11 months ago Modified 1 year, 3 months ago Viewed 82k times

331

This question is motivated by my another question: [How to await in cdef?]

There are tons of articles and blog posts on the web about `asyncio`, but they are all very superficial. I couldn't find any information about how `asyncio` is actually implemented, and what makes I/O asynchronous. I was trying to read the source code, but it's thousands of lines of not the highest grade C code, a lot of which deals with auxiliary objects, but most crucially, it is hard to connect between Python syntax and what C code it would translate into.

Asycnio's own documentation is even less helpful. There's no information there about how it works, only some guidelines about how to use it, which are also sometimes misleading / very poorly written.

I'm familiar with Go's implementation of coroutines, and was kind of hoping that Python did the same thing. If that was the case, the code I came up in the post linked above would have worked. Since it didn't, I'm now trying to figure out why. My best guess so far is as follows, please correct me where I'm wrong:

1. Procedure definitions of the form `async def foo(): ...` are actually interpreted as methods of a class inheriting `coroutine`.
2. Perhaps, `async def` is actually split into multiple methods by `await` statements, where the object, on which these methods are called is able to keep track of the progress it made through the execution so far.
3. If the above is true, then, essentially, execution of a coroutine boils down to calling methods of coroutine object by some global manager (loop?).
4. The global manager is somehow (how?) aware of when I/O operations are performed by Python (only?) code and is able to choose one of the pending coroutine methods to execute after the current executing method relinquished control (hit on the `await` statement).

In other words, here's my attempt at "desugaring" of some `asyncio` syntax into something more understandable:

```
async def coro(name):
    print('before', name)
    await asyncio.sleep()
    print('after', name)

asyncio.gather(coro('first'), coro('second'))

# translated from async def coro(name)
class Coro(coroutine):
    def before(self, name):
        print('before', name)
```

```
    def after(self, name):
        print('after', name)

    def __init__(self, name):
        self.name = name
        self.parts = self.before, self.after
        self.pos = 0

    def __call__():
        self.parts[self.pos](self.name)
        self.pos += 1

    def done(self):
        return self.pos == len(self.parts)


# translated from asyncio.gather()
class AsyncIOManager:

    def gather(*coros):
        while not every(c.done() for c in coros):
            coro = random.choice(coros)
            coro()
```

Should my guess prove correct: then I have a problem. How does I/O actually happen in this scenario? In a separate thread? Is the whole interpreter suspended and I/O happens outside the interpreter? What exactly is meant by I/O? If my python procedure called C `open()` procedure, and it in turn sent interrupt to kernel, relinquishing control to it, how does Python interpreter know about this and is able to continue running some other code, while kernel code does the actual I/O and until it wakes up the Python procedure which sent the interrupt originally? How can Python interpreter in principle, be aware of this happening?

python python-3.x python-asyncio python-internals

Share Improve this question Follow

edited Jan 27, 2022 at 14:55
Bharel
24.5k 6 44 85

asked Feb 27, 2018 at 9:48
wvxvw
8,486 10 34 66

3 Most of the logic is handled by the event loop implementation. Look at how the CPython `BaseEventLoop` is implemented: github.com/python/cpython/blob/... – Blender Feb 27, 2018 at 17:42

1 @Blender ok, I think I finally found what I wanted, but now I don't understand the reason the code was written the way it was. Why is `_run_once`, which is actually the only useful function in this entire module made "private"? The implementation is horrible, but that's less of a problem. Why is the only function you would ever want to call on event loop is marked as "don't call me"? – wvxvw Feb 28, 2018 at 7:02

That's a question for the mailing list. What use case would require you to touch `_run_once` in the first place? – Blender Feb 28, 2018 at 7:46

14 That doesn't really answer my question, though. How would you solve any useful problem using just `_run_once`? `asyncio` is complex and has its faults, but please keep the discussion civil. Don't badmouth

the developers behind code that you yourself don't understand. – Blender Feb 28, 2018 at 18:03

1 Note that *only* `async def` , `async for` , `async with` and `await` are part of the Python spec. These are basically classical generator based coroutines (`def ...: yield`) and their temporary delegation (`yield from ...`), extending the later to `with` and `for` . In contrast `asyncio` is an I/O library built using those primitives. There are other libraries, such as `curio` and `trio` - mainly built to be easier to reason about. You may want to check them out after reading @Bharel 's answer. – MisterMiyagi Jul 3, 2018 at 11:52

6 Answers

Sorted by: Highest score (default)

588

+1300

How does asyncio work?

Before answering this question we need to understand a few base terms, skip these if you already know any of them.

Generators

Generators are objects that allow us to suspend the execution of a python function. User curated generators are implemented using the keyword `yield` . By creating a normal function containing the `yield` keyword, we turn that function into a generator:

```
>>> def test():
...     yield 1
...     yield 2
...
>>> gen = test()
>>> next(gen)
1
>>> next(gen)
2
>>> next(gen)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

As you can see, calling `next()` on the generator causes the interpreter to load the test's frame, and return the `yield` ed value. Calling `next()` again, causes the frame to load again into the interpreter stack, and continues on `yield` ing another value.

By the third time `next()` is called, our generator was finished, and `StopIteration` was thrown.

Communicating with a generator

A less-known feature of generators is the fact that you can communicate with them using two methods: `send()` and `throw()` .

```
>>> def test():
...     val = yield 1
...     print(val)
...     yield 2
...     yield 3
...
>>> gen = test()
>>> next(gen)
1
>>> gen.send("abc")
abc
2
>>> gen.throw(Exception())
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in test
Exception
```

Upon calling `gen.send()`, the value is passed as a return value from the `yield` keyword.

`gen.throw()` on the other hand, allows throwing Exceptions inside generators, with the exception raised at the same spot `yield` was called.

Returning values from generators

Returning a value from a generator, results in the value being put inside the `StopIteration` exception. We can later on recover the value from the exception and use it to our needs.

```
>>> def test():
...     yield 1
...     return "abc"
...
>>> gen = test()
>>> next(gen)
1
>>> try:
...     next(gen)
... except StopIteration as exc:
...     print(exc.value)
...
abc
```

Behold, a new keyword: `yield from`

Python 3.4 came with the addition of a new keyword: `yield from`. What that keyword allows us to do, is pass on any `next()`, `send()` and `throw()` into an inner-most nested generator. If the inner generator returns a value, it is also the return value of `yield from`:

```
>>> def inner():
...     inner_result = yield 2
...     print('inner', inner_result)
...     return 3
```

```
...
>>> def outer():
...     yield 1
...     val = yield from inner()
...     print('outer', val)
...     yield 4
...
>>> gen = outer()
>>> next(gen)
1
>>> next(gen) # Goes inside inner() automatically
2
>>> gen.send("abc")
inner abc
outer 3
4
```

I've written an article to further elaborate on this topic.

Putting it all together

Upon introducing the new keyword `yield from` in Python 3.4, we were now able to create generators inside generators that just like a tunnel, pass the data back and forth from the inner-most to the outer-most generators. This has spawned a new meaning for generators - *coroutines*.

Coroutines are functions that can be stopped and resumed while being run. In Python, they are defined using the `async def` keyword. Much like generators, they too use their own form of `yield from` which is `await`. Before `async` and `await` were introduced in Python 3.5, we created coroutines in the exact same way generators were created (with `yield from` instead of `await`).

```
async def inner():
    return 1

async def outer():
    await inner()
```

Just like all iterators and generators implement the `__iter__()` method, all coroutines implement `__await__()` which allows them to continue on every time `await coro` is called.

There's a nice sequence diagram inside the Python docs that you should check out.

In asyncio, apart from coroutine functions, we have 2 important objects: **tasks** and **futures**.

Futures

Futures are objects that have the `__await__()` method implemented, and their job is to hold a certain state and result. The state can be one of the following:

1. PENDING - future does not have any result or exception set.

2. CANCELLED - future was cancelled using `fut.cancel()`
3. FINISHED - future was finished, either by a result set using `fut.set_result()` or by an exception set using `fut.set_exception()`

The result, just like you have guessed, can either be a Python object, that will be returned, or an exception which may be raised.

Another **important** feature of `future` objects, is that they contain a method called `add_done_callback()`. This method allows functions to be called as soon as the task is done - whether it raised an exception or finished.

Tasks

Task objects are special futures, which wrap around coroutines, and communicate with the inner-most and outer-most coroutines. Every time a coroutine `await` s a future, the future is passed all the way back to the task (just like in `yield from`), and the task receives it.

Next, the task binds itself to the future. It does so by calling `add_done_callback()` on the future. From now on, if the future will ever be done, by either being cancelled, passed an exception or passed a Python object as a result, the task's callback will be called, and it will rise back up to existence.

Asyncio

The final burning question we must answer is - how is the IO implemented?

Deep inside asyncio, we have an event loop. An event loop of tasks. The event loop's job is to call tasks every time they are ready and coordinate all that effort into one single working machine.

The IO part of the event loop is built upon a single crucial function called `select`. Select is a blocking function, implemented by the operating system underneath, that allows waiting on sockets for incoming or outgoing data. Upon receiving data it wakes up, and returns the sockets which received data, or the sockets which are ready for writing.

When you try to receive or send data over a socket through asyncio, what actually happens below is that the socket is first checked if it has any data that can be immediately read or sent. If its `.send()` buffer is full, or the `.recv()` buffer is empty, the socket is registered to the `select` function (by simply adding it to one of the lists, `rlist` for `recv` and `wlist` for `send`) and the appropriate function `await` s a newly created `future` object, tied to that socket.

When all available tasks are waiting for futures, the event loop calls `select` and waits. When the one of the sockets has incoming data, or its `send` buffer drained up, asyncio checks for the future object tied to that socket, and sets it to done.

Now all the magic happens. The future is set to done, the task that added itself before with `add_done_callback()` rises up back to life, and calls `.send()` on the coroutine which resumes the inner-most coroutine (because of the `await` chain) and you read the newly received data from a nearby buffer it was spilled unto.

Method chain again, in case of `recv()`:

1. `select.select` waits.
2. A ready socket, with data is returned.
3. Data from the socket is moved into a buffer.
4. `future.set_result()` is called.
5. Task that added itself with `add_done_callback()` is now woken up.
6. Task calls `.send()` on the coroutine which goes all the way into the inner-most coroutine and wakes it up.
7. Data is being read from the buffer and returned to our humble user.

In summary, asyncio uses generator capabilities, that allow pausing and resuming functions. It uses `yield from` capabilities that allow passing data back and forth from the inner-most generator to the outer-most. It uses all of those in order to halt function execution while it's waiting for IO to complete (by using the OS `select` function).

And the best of all? While one function is paused, another may run and interleave with the delicate fabric, which is asyncio.

Share Improve this answer Follow

edited Oct 5, 2022 at 18:51
Andy
3 2

answered Jun 30, 2018 at 17:03
Bharel
24.5k 6 44 85

45 If there's any more explanation needed, don't hesitate to comment. Btw, I'm not entirely sure if I should have written this as a blog article or an answer in stackoverflow. The question is a long one to answer. – Bharel Jun 30, 2018 at 18:43

6 @user8371915 Always here to help :-) Keep in mind that in order to understand Asyncio you *must* know how generators, generator communication and `yield from` works. I did however note up top that it's skippable in case the reader already knows about it :-) Anything else you believe I should add? – Bharel Jul 2, 2018 at 19:36

7 The things before the **Asyncio** section are perhaps the most critical, as they are the only thing the language actually does by itself. The `select` may qualify as well, since it is how non-blocking I/O system calls work on OS. The actual `asyncio` constructs and event loop are just app-level code built from these things. – MisterMiyagi Jul 3, 2018 at 14:47

7 This post has info of backbone of asynchronous I/O in Python. Thanks for such a kind explanation. – mjkim Sep 16, 2018 at 15:22

6 @zonk 1. Event loop checks futures, and if nothing happens waits on sockets for data. 2. Multiple tasks are thrown into an internal list that acts as a fifo queue. Only tasks that are ready for execution are thrown in

there. 3. Each cycle all tasks that are ready are executed, and ready sockets awake their relevant tasks. If a task waits for a socket, it won't be added to the queue, and as soon as the socket will be ready, the event loop will add it back. 4. Event loop does not guarantee order. Besides, apart from 2 sockets, there can't be 2 tasks that finish on the same time (Single thread). – Bharel Feb 17, 2020 at 15:56

215

+300

Talking about `async/await` and `asyncio` is not the same thing. The first is a fundamental, low-level construct (coroutines) while the later is a library using these constructs. Conversely, there is no single ultimate answer.

The following is a general description of how `async/await` and `asyncio` -like libraries work. That is, there may be other tricks on top (there are...) but they are inconsequential unless you build them yourself. The difference should be negligible unless you already know enough to not have to ask such a question.

1. Coroutines versus subroutines in a nut shell

Just like **subroutines** (functions, procedures, ...), **coroutines** (generators, ...) are an abstraction of call stack and instruction pointer: there is a stack of executing code pieces, and each is at a specific instruction.

The distinction of `def` versus `async def` is merely for clarity. The actual difference is `return` versus `yield`. From this, `await` or `yield from` take the difference from individual calls to entire stacks.

1.1. Subroutines

A subroutine represents a new stack level to hold local variables, and a single traversal of its instructions to reach an end. Consider a subroutine like this:

```
def subfoo(bar):
    qux = 3
    return qux * bar
```

When you run it, that means

1. allocate stack space for `bar` and `qux`
2. recursively execute the first statement and jump to the next statement
3. once at a `return`, push its value to the calling stack
4. clear the stack (1.) and instruction pointer (2.)

Notably, 4. means that a subroutine always starts at the same state. Everything exclusive to the function itself is lost upon completion. A function cannot be resumed, even if there are instructions after `return`.

```
root -\
  :    \- subfoo --\
  :/--<---return --/
  |
  V
```

1.2. Coroutines as persistent subroutines

A coroutine is like a subroutine, but can exit *without* destroying its state. Consider a coroutine like this:

```
def cofoo(bar):
     qux = yield bar  # yield marks a break point
     return qux
```

When you run it, that means

1. allocate stack space for `bar` and `qux`
2. recursively execute the first statement and jump to the next statement
    1. once at a `yield`, push its value to the calling stack *but store the stack and instruction pointer*
    2. once calling into `yield`, restore stack and instruction pointer and push arguments to `qux`
3. once at a `return`, push its value to the calling stack
4. clear the stack (1.) and instruction pointer (2.)

Note the addition of 2.1 and 2.2 - a coroutine can be suspended and resumed at predefined points. This is similar to how a subroutine is suspended during calling another subroutine. The difference is that the active coroutine is not strictly bound to its calling stack. Instead, a suspended coroutine is part of a separate, isolated stack.

```
root -\
  :    \- cofoo --\
  :/--<+--yield --/
  |    :
  V    :
```

This means that suspended coroutines can be freely stored or moved between stacks. Any call stack that has access to a coroutine can decide to resume it.

1.3. Traversing the call stack

So far, our coroutine only goes down the call stack with `yield`. A subroutine can go down *and up* the call stack with `return` and `()`. For completeness, coroutines also need a mechanism to go up

the call stack. Consider a coroutine like this:

```
def wrap():
    yield 'before'
    yield from cofoo()
    yield 'after'
```

When you run it, that means it still allocates the stack and instruction pointer like a subroutine. When it suspends, that still is like storing a subroutine.

However, `yield from` does *both*. It suspends stack and instruction pointer of `wrap` *and* runs `cofoo`. Note that `wrap` stays suspended until `cofoo` finishes completely. Whenever `cofoo` suspends or something is sent, `cofoo` is directly connected to the calling stack.

1.4. Coroutines all the way down

As established, `yield from` allows to connect two scopes across another intermediate one. When applied recursively, that means the *top* of the stack can be connected to the *bottom* of the stack.

```
root -\
  :    \-> coro_a -yield-from-> coro_b --\
  :/ <-+------------------------yield ---/
  |    :
  :\ --+-- coro_a.send----------yield ---\
  :                             coro_b <-/
```

Note that `root` and `coro_b` do not know about each other. This makes coroutines much cleaner than callbacks: coroutines still built on a 1:1 relation like subroutines. Coroutines suspend and resume their entire existing execution stack up until a regular call point.

Notably, `root` could have an arbitrary number of coroutines to resume. Yet, it can never resume more than one at the same time. Coroutines of the same root are concurrent but not parallel!

1.5. Python's `async` and `await`

The explanation has so far explicitly used the `yield` and `yield from` vocabulary of generators - the underlying functionality is the same. The new Python3.5 syntax `async` and `await` exists mainly for clarity.

```
def foo():  # subroutine?
     return None

def foo():  # coroutine?
     yield from foofoo()  # generator? coroutine?

async def foo():  # coroutine!
```

```
    await foofoo()  # coroutine!
    return None
```

The `async for` and `async with` statements are needed because you would break the `yield from/await` chain with the bare `for` and `with` statements.

2. Anatomy of a simple event loop

By itself, a coroutine has no concept of yielding control to *another* coroutine. It can only yield control to the caller at the bottom of a coroutine stack. This caller can then switch to another coroutine and run it.

This root node of several coroutines is commonly an **event loop**: on suspension, a coroutine yields an **event** on which it wants resume. In turn, the event loop is capable of efficiently waiting for these events to occur. This allows it to decide which coroutine to run next, or how to wait before resuming.

Such a design implies that there is a set of pre-defined events that the loop understands. Several coroutines `await` each other, until finally an event is `await` ed. This event can communicate *directly* with the event loop by `yield` ing control.

```
loop -\
  :    \-> coroutine --await--> event --\
  :/ <-+----------------------- yield --/
  |    :
  |    :  # loop waits for event to happen
  |    :
  :\ --+-- send(reply) -------- yield --\
  :        coroutine <--yield-- event <-/
```

The key is that coroutine suspension allows the event loop and events to directly communicate. The intermediate coroutine stack does not require *any* knowledge about which loop is running it, nor how events work.

2.1.1. Events in time

The simplest event to handle is reaching a point in time. This is a fundamental block of threaded code as well: a thread repeatedly `sleep` s until a condition is true. However, a regular `sleep` blocks execution by itself - we want other coroutines to not be blocked. Instead, we want tell the event loop when it should resume the current coroutine stack.

2.1.2. Defining an Event

An event is simply a value we can identify - be it via an enum, a type or other identity. We can define this with a simple class that stores our target time. In addition to *storing* the event information, we can allow to `await` a class directly.

```
class AsyncSleep:
    """Event to sleep until a point in time"""
    def __init__(self, until: float):
        self.until = until

    # used whenever someone ``await``s an instance of this Event
    def __await__(self):
        # yield this Event to the loop
        yield self

    def __repr__(self):
        return '%s(until=%.1f)' % (self.__class__.__name__, self.until)
```

This class only *stores* the event - it does not say how to actually handle it.

The only special feature is `__await__` - it is what the `await` keyword looks for. Practically, it is an iterator but not available for the regular iteration machinery.

2.2.1. Awaiting an event

Now that we have an event, how do coroutines react to it? We should be able to express the equivalent of `sleep` by `await` ing our event. To better see what is going on, we wait twice for half the time:

```
import time

async def asleep(duration: float):
    """await that ``duration`` seconds pass"""
    await AsyncSleep(time.time() + duration / 2)
    await AsyncSleep(time.time() + duration / 2)
```

We can directly instantiate and run this coroutine. Similar to a generator, using `coroutine.send` runs the coroutine until it `yield` s a result.

```
coroutine = asleep(100)
while True:
    print(coroutine.send(None))
    time.sleep(0.1)
```

This gives us two `AsyncSleep` events and then a `StopIteration` when the coroutine is done. Notice that the only delay is from `time.sleep` in the loop! Each `AsyncSleep` only stores an offset from the current time.

2.2.2. Event + Sleep

At this point, we have *two* separate mechanisms at our disposal:

- `AsyncSleep` Events that can be yielded from inside a coroutine
- `time.sleep` that can wait without impacting coroutines

Notably, these two are orthogonal: neither one affects or triggers the other. As a result, we can come up with our own strategy to `sleep` to meet the delay of an `AsyncSleep`.

2.3. A naive event loop

If we have *several* coroutines, each can tell us when it wants to be woken up. We can then wait until the first of them wants to be resumed, then for the one after, and so on. Notably, at each point we only care about which one is *next*.

This makes for a straightforward scheduling:

1. sort coroutines by their desired wake up time
2. pick the first that wants to wake up
3. wait until this point in time
4. run this coroutine
5. repeat from 1.

A trivial implementation does not need any advanced concepts. A `list` allows to sort coroutines by date. Waiting is a regular `time.sleep`. Running coroutines works just like before with `coroutine.send`.

```
def run(*coroutines):
    """Cooperatively run all ``coroutines`` until completion"""
    # store wake-up-time and coroutines
    waiting = [(0, coroutine) for coroutine in coroutines]
    while waiting:
        # 2. pick the first coroutine that wants to wake up
        until, coroutine = waiting.pop(0)
        # 3. wait until this point in time
        time.sleep(max(0.0, until - time.time()))
        # 4. run this coroutine
        try:
            command = coroutine.send(None)
        except StopIteration:
            continue
        # 1. sort coroutines by their desired suspension
        if isinstance(command, AsyncSleep):
            waiting.append((command.until, coroutine))
            waiting.sort(key=lambda item: item[0])
```

Of course, this has ample room for improvement. We can use a heap for the wait queue or a dispatch table for events. We could also fetch return values from the `StopIteration` and assign them to the coroutine. However, the fundamental principle remains the same.

2.4. Cooperative Waiting

The `AsyncSleep` event and `run` event loop are a fully working implementation of timed events.

```
async def sleepy(identifier: str = "coroutine", count=5):
    for i in range(count):
        print(identifier, 'step', i + 1, 'at %.2f' % time.time())
        await asleep(0.1)

run(*(sleepy("coroutine %d" % j) for j in range(5)))
```

This cooperatively switches between each of the five coroutines, suspending each for 0.1 seconds. Even though the event loop is synchronous, it still executes the work in 0.5 seconds instead of 2.5 seconds. Each coroutine holds state and acts independently.

3. I/O event loop

An event loop that supports `sleep` is suitable for *polling*. However, waiting for I/O on a file handle can be done more efficiently: the operating system implements I/O and thus knows which handles are ready. Ideally, an event loop should support an explicit "ready for I/O" event.

3.1. The `select` call

Python already has an interface to query the OS for read I/O handles. When called with handles to read or write, it returns the handles *ready* to read or write:

```
readable, writable, _ = select.select(rlist, wlist, xlist, timeout)
```

For example, we can `open` a file for writing and wait for it to be ready:

```
write_target = open('/tmp/foo')
readable, writable, _ = select.select([], [write_target], [])
```

Once select returns, `writable` contains our open file.

3.2. Basic I/O event

Similar to the `AsyncSleep` request, we need to define an event for I/O. With the underlying `select` logic, the event must refer to a readable object - say an `open` file. In addition, we store how much data to read.

```
class AsyncRead:
    def __init__(self, file, amount=1):
        self.file = file
        self.amount = amount
        self._buffer = b'' if 'b' in file.mode else ''

    def __await__(self):
        while len(self._buffer) < self.amount:
            yield self
            # we only get here if ``read`` should not block
            self._buffer += self.file.read(1)
        return self._buffer

    def __repr__(self):
        return '%s(file=%s, amount=%d, progress=%d)' % (
            self.__class__.__name__, self.file, self.amount, len(self._buffer)
        )
```

As with `AsyncSleep` we mostly just store the data required for the underlying system call. This time, `__await__` is capable of being resumed multiple times - until our desired `amount` has been read. In addition, we `return` the I/O result instead of just resuming.

3.3. Augmenting an event loop with read I/O

The basis for our event loop is still the `run` defined previously. First, we need to track the read requests. This is no longer a sorted schedule, we only map read requests to coroutines.

```
# new
waiting_read = {}  # type: Dict[file, coroutine]
```

Since `select.select` takes a timeout parameter, we can use it in place of `time.sleep`.

```
# old
time.sleep(max(0.0, until - time.time()))
# new
readable, _, _ = select.select(list(waiting_read), [], [])
```

This gives us all readable files - if there are any, we run the corresponding coroutine. If there are none, we have waited long enough for our current coroutine to run.

```
# new - reschedule waiting coroutine, run readable coroutine
if readable:
    waiting.append((until, coroutine))
    waiting.sort()
    coroutine = waiting_read[readable[0]]
```

Finally, we have to actually listen for read requests.

```
# new
if isinstance(command, AsyncSleep):
    ...
elif isinstance(command, AsyncRead):
    ...
```

3.4. Putting it together

The above was a bit of a simplification. We need to do some switching to not starve sleeping coroutines if we can always read. We need to handle having nothing to read or nothing to wait for. However, the end result still fits into 30 LOC.

```
def run(*coroutines):
    """Cooperatively run all ``coroutines`` until completion"""
    waiting_read = {}  # type: Dict[file, coroutine]
    waiting = [(0, coroutine) for coroutine in coroutines]
    while waiting or waiting_read:
        # 2. wait until the next coroutine may run or read ...
        try:
            until, coroutine = waiting.pop(0)
        except IndexError:
            until, coroutine = float('inf'), None
            readable, _, _ = select.select(list(waiting_read), [], [])
        else:
            readable, _, _ = select.select(list(waiting_read), [], [], max(0.0, until -
time.time()))
        # ... and select the appropriate one
        if readable and time.time() < until:
            if until and coroutine:
                waiting.append((until, coroutine))
                waiting.sort()
            coroutine = waiting_read.pop(readable[0])
        # 3. run this coroutine
        try:
            command = coroutine.send(None)
        except StopIteration:
            continue
        # 1. sort coroutines by their desired suspension ...
        if isinstance(command, AsyncSleep):
            waiting.append((command.until, coroutine))
            waiting.sort(key=lambda item: item[0])
        # ... or register reads
        elif isinstance(command, AsyncRead):
            waiting_read[command.file] = coroutine
```

3.5. Cooperative I/O

The `AsyncSleep`, `AsyncRead` and `run` implementations are now fully functional to sleep and/or read. Same as for `sleepy`, we can define a helper to test reading:

```
async def ready(path, amount=1024*32):
    print('read', path, 'at', '%d' % time.time())
    with open(path, 'rb') as file:
        result = await AsyncRead(file, amount)
    print('done', path, 'at', '%d' % time.time())
    print('got', len(result), 'B')

run(sleepy('background', 5), ready('/dev/urandom'))
```

Running this, we can see that our I/O is interleaved with the waiting task:

```
id background round 1
read /dev/urandom at 1530721148
id background round 2
id background round 3
id background round 4
id background round 5
done /dev/urandom at 1530721148
got 1024 B
```

4. Non-Blocking I/O

While I/O on files gets the concept across, it is not really suitable for a library like `asyncio`: the `select` call [always returns for files](), and both `open` and `read` may [block indefinitely](). This blocks all coroutines of an event loop - which is bad. Libraries like `aiofiles` use threads and synchronization to fake non-blocking I/O and events on file.

However, sockets do allow for non-blocking I/O - and their inherent latency makes it much more critical. When used in an event loop, waiting for data and retrying can be wrapped without blocking anything.

4.1. Non-Blocking I/O event

Similar to our `AsyncRead`, we can define a suspend-and-read event for sockets. Instead of taking a file, we take a socket - which must be non-blocking. Also, our `__await__` uses `socket.recv` instead of `file.read`.

```
class AsyncRecv:
    def __init__(self, connection, amount=1, read_buffer=1024):
        assert not connection.getblocking(), 'connection must be non-blocking for async
recv'
        self.connection = connection
        self.amount = amount
        self.read_buffer = read_buffer
        self._buffer = b''

    def __await__(self):
        while len(self._buffer) < self.amount:
            try:
                self._buffer += self.connection.recv(self.read_buffer)
```

```
            except BlockingIOError:
                yield self
        return self._buffer

    def __repr__(self):
        return '%s(file=%s, amount=%d, progress=%d)' % (
            self.__class__.__name__, self.connection, self.amount, len(self._buffer)
        )
```

In contrast to `AsyncRead`, `__await__` performs truly non-blocking I/O. When data is available, it *always* reads. When no data is available, it *always* suspends. That means the event loop is only blocked while we perform useful work.

4.2. Un-Blocking the event loop

As far as the event loop is concerned, nothing changes much. The event to listen for is still the same as for files - a file descriptor marked ready by `select`.

```
# old
elif isinstance(command, AsyncRead):
    waiting_read[command.file] = coroutine
# new
elif isinstance(command, AsyncRead):
    waiting_read[command.file] = coroutine
elif isinstance(command, AsyncRecv):
    waiting_read[command.connection] = coroutine
```

At this point, it should be obvious that `AsyncRead` and `AsyncRecv` are the same kind of event. We could easily refactor them to be *one* event with an exchangeable I/O component. In effect, the event loop, coroutines and events cleanly separate a scheduler, arbitrary intermediate code and the actual I/O.

4.3. The ugly side of non-blocking I/O

In principle, what you should do at this point is replicate the logic of `read` as a `recv` for `AsyncRecv`. However, this is much more ugly now - you have to handle early returns when functions block inside the kernel, but yield control to you. For example, opening a connection versus opening a file is much longer:

```
# file
file = open(path, 'rb')
# non-blocking socket
connection = socket.socket()
connection.setblocking(False)
# open without blocking - retry on failure
try:
    connection.connect((url, port))
except BlockingIOError:
    pass
```

Long story short, what remains is a few dozen lines of Exception handling. The events and event loop already work at this point.

```
id background round 1
read localhost:25000 at 1530783569
read /dev/urandom at 1530783569
done localhost:25000 at 1530783569 got 32768 B
id background round 2
id background round 3
id background round 4
done /dev/urandom at 1530783569 got 4096 B
id background round 5
```

Addendum

[Example code at github]

Share Improve this answer Follow edited Jul 27, 2022 at 7:33 answered Jul 4, 2018 at 16:30

MisterMiyagi
46.1k 12 109 125

Using `yield self` in AsyncSleep gives me `Task got back yield` error, why is that? I do see that the code in asyncio.Futures uses that. Using a bare yield works fine. – Ron Serruya Jun 26, 2019 at 17:55

1 Event loops usually only expect their own events. You generally cannot mix events and event loops across libraries; the events shown here only work with the event loop shown. In specific, asyncio only uses None (i.e. a bare yield) as a signal for the event loop. Events directly interact with the event loop object to register wakeups. – MisterMiyagi Jun 26, 2019 at 18:21

1 So asyncio is in fact polling on every "loop iteration" because of the call to recv, even though the call might not be expensive enough to really be a problem. Wouldn't it make sense to implement an I/O OS thread for every socket in the asyncio module written in C so that this thread can wait without any cost (through select, epoll etc.), or would that add disproportional overhead compared to polling between function call events etc.? – Jonniboy Jan 12, 2022 at 17:32

@S.B Any reason for the rollback? Your changes looked fine. – MisterMiyagi Jul 27, 2022 at 7:17

1 @S.B Thanks for spotting it and applying a first fix. I've just swapped that part with detecting the mode and accepted all other changes. – MisterMiyagi Jul 27, 2022 at 7:34

20

Your `coro` desugaring is conceptually correct, but slightly incomplete.

`await` doesn't suspend unconditionally, but only if it encounters a blocking call. How does it know that a call is blocking? This is decided by the code being awaited. For example, an awaitable implementation of socket read could be desugared to:

```
def read(sock, n):
    # sock must be in non-blocking mode
```

```
    try:
        return sock.recv(n)
    except EWOULDBLOCK:
        event_loop.add_reader(sock.fileno, current_task())
        return SUSPEND
```

In real asyncio the equivalent code modifies the state of a `Future` instead of returning magic values, but the concept is the same. When appropriately adapted to a generator-like object, the above code can be `await` ed.

On the caller side, when your coroutine contains:

```
data = await read(sock, 1024)
```

It desugars into something close to:

```
data = read(sock, 1024)
if data is SUSPEND:
    return SUSPEND
self.pos += 1
self.parts[self.pos](...)
```

People familiar with generators tend to describe the above in terms of `yield from` which does the suspension automatically.

The suspension chain continues all the way up to the event loop, which notices that the coroutine is suspended, removes it from the runnable set, and goes on to execute coroutines that are runnable, if any. If no coroutines are runnable, the loop waits in `select()` until either a file descriptor a coroutine is interested in becomes ready for IO or a timeout expires. (The event loop maintains a file-descriptor-to-coroutine mapping.)

In the above example, once `select()` tells the event loop that `sock` is readable, it will re-add `coro` to the runnable set, so it will be continued from the point of suspension.

In other words:

1. *Everything* happens in the same thread by default.
2. The event loop is responsible for scheduling the coroutines and waking them up when whatever they were waiting for (typically an IO call that would normally block, or a timeout) becomes ready.

For insight on coroutine-driving event loops, I recommend this talk by Dave Beazley, where he demonstrates coding an event loop from scratch in front of live audience.

Share Improve this answer Follow edited Jan 14, 2021 at 18:01 answered Feb 27, 2018 at 17:04

user4815162342
146k 18 309 370

Thank you, this is closer to what I'm after, but, this still doesn't explain why `async.wait_for()` doesn't do what it's supposed to... Why is it such a big problem to add a callback to event loop and tell it to process however many callbacks it needs, including the one you just added? My frustration with `asyncio` is in part due to the fact that the underlying concept is very simple, and, for example, Emacs Lisp had implementation for ages, without using buzzwords... (i.e. `create-async-process` and `accept-process-output` - and this is all that is needed... (contd) – wvxvw Feb 28, 2018 at 6:37

14 @wvxvw I've done as much as I could to answer the question you posted, as much as that's even possible given that only the last paragraph contains six questions. And so we go on - it's not that `wait_for` *doesn't do what it's supposed to* (it does, it's a coroutine you're supposed to await), it's that your expectations don't match what the system was designed and implemented to do. I think your problem could be matched to asyncio if the event loop were running in a separate thread, but I don't know the details of your use case and, honestly, your attitude doesn't make it much fun to help you. – user4815162342 Feb 28, 2018 at 8:30

7 @wvxvw `My frustration with asyncio is in part due to the fact that the underlying concept is very simple, and, for example, Emacs Lisp had implementation for ages, without using buzzwords...` - Nothing stops you from implementing this simple concept without buzzwords for the Python then :) Why do you use this ugly asyncio at all? Implement your own from scratch. For example, you can start with creating your own `async.wait_for()` function that does exactly what it supposed to. – Mikhail Gerasimov Feb 28, 2018 at 8:50

1 @MikhailGerasimov you seem to think it's a rhetorical question. But, I'd like to dispel the mystery for you. Language is designed to speak to others. I cannot choose for others which language they speak, even if I believe the language they speak is garbage, the best I can do is to try to convince them that is the case. In other words, if I was free to choose, I'd never choose Python to begin with, let alone `asyncio`. But, in principle, that is not my decision to make. I'm coerced into using garbage language through en.wikipedia.org/wiki/Ultimatum_game . – wvxvw Feb 28, 2018 at 9:45

1 @wvxvw: but, if you're *not* free to choose, it still behooves you as a professional to do the best job you can with the tools provided. In *this* case, that probably just means, however inconvenient for you, adapting to `asyncio` :-) But, by all means, keep trying to convince the powers that be that a non-Python solution is better (if that truly is the case) - I was forced to use Powershell under Windows but eventually managed to convince the powers that Python was better, mostly so we could avoid costly duplication of effort (we had to script Windows *and* embedded Linux). – paxdiablo Jul 8, 2021 at 2:59

7

It all boils down to the two main challenges that asyncio is addressing:

- How to perform multiple I/O in a single thread?
- How to implement cooperative multitasking?

The answer to the first point has been around for a long while and is called a select loop. In python, it is implemented in the selectors module.

The second question is related to the concept of coroutine, i.e. functions that can stop their execution and be restored later on. In python, coroutines are implemented using generators and the yield from statement. That's what is hiding behind the async/await syntax.

More resources in this answer.

EDIT: Addressing your comment about goroutines:

The closest equivalent to a goroutine in asyncio is actually not a coroutine but a task (see the difference in the documentation). In python, a coroutine (or a generator) knows nothing about the concepts of event loop or I/O. It simply is a function that can stop its execution using `yield` while keeping its current state, so it can be restored later on. The `yield from` syntax allows for chaining them in a transparent way.

Now, within an asyncio task, the coroutine at the very bottom of the chain always ends up yielding a future. This future then bubbles up to the event loop, and gets integrated into the inner machinery. When the future is set to done by some other inner callback, the event loop can restore the task by sending the future back into the coroutine chain.

EDIT: Addressing some of the questions in your post:

> How does I/O actually happen in this scenario? In a separate thread? Is the whole interpreter suspended and I/O happens outside the interpreter?

No, nothing happens in a thread. I/O is always managed by the event loop, mostly through file descriptors. However the registration of those file descriptors is usually hidden by high-level coroutines, making the dirty work for you.

> What exactly is meant by I/O? If my python procedure called C open() procedure, and it in turn sent interrupt to kernel, relinquishing control to it, how does Python interpreter know about this and is able to continue running some other code, while kernel code does the actual I/O and until it wakes up the Python procedure which sent the interrupt originally? How can Python interpreter in principle, be aware of this happening?

An I/O is any blocking call. In asyncio, all the I/O operations should go through the event loop, because as you said, the event loop has no way to be aware that a blocking call is being performed in some synchronous code. That means you're not supposed to use a synchronous `open` within the context of a coroutine. Instead, use a dedicated library such aiofiles which provides an asynchronous version of `open`.

Share Improve this answer Follow edited Feb 27, 2018 at 15:37 answered Feb 27, 2018 at 11:11

Vincent
13.1k 1 44 64

> Saying that coroutines are implemented using `yield from` doesn't really say anything. `yield from` is just a syntax construct, it isn't a fundamental building block that computers can execute. Similarly, for select

loop. Yes, coroutines in Go also use select loop, but what I was trying to do would work in Go, but doesn't in Python. I need more detailed answers to understand why it didn't work. – wvxvw Feb 27, 2018 at 11:24

Sorry... no, not really. "future", "task", "transparent way", "yield from" are just buzzwords, they aren't objects from domain of programming. programming has variables, procedures and structures. So, to say that "goroutine is a task" is just a circular statement that begs a question. Ultimately, an explanation of what `asyncio` does, for me, would boil down to C code that illustrates what Python syntax got translated into. – wvxvw Feb 27, 2018 at 14:59

To further explain why your answer doesn't answer my question: with all the information you provided, I have no idea as to why my attempt from the code I posted in the linked question didn't work. I'm absolutely certain that I could write event loop in such a way that this code would work. In fact, this would be the way I'd write an event loop, if I had to write one. – wvxvw Feb 27, 2018 at 15:01

9 @wvxvw I disagree. Those are not "buzzwords" but high-level concepts that have been implemented in many libraries. For instance, an asyncio task, a gevent greenlet and a goroutine all correspond to the the same thing: an execution unit that can run concurrently within a single thread. Also I don't think C is needed to understand asyncio at all, unless you want to get into the inner workings of python generators. – Vincent Feb 27, 2018 at 15:16

@wvxvw See my second edit. This should clear a few misconceptions out of the way. – Vincent Feb 27, 2018 at 15:39

1

If you picture an airport control tower, with many planes waiting to land on the same runway. The control tower can be seen as the event loop and runway as the thread. Each plane is a separate function waiting to execute. In reality only one plane can land on the runway at a time. What asyncio basically does it allows many planes to land simultaneously on the same runway by using the event loop to suspend functions and allow other functions to run when you use the await syntax it basically means that plane(function can be suspended and allow other functions to process

Share Improve this answer Follow

answered Oct 20, 2022 at 9:14

Foton
97 1 9

0

It allows you to write single-threaded asynchronous code and implement concurrency in Python. Basically, `asyncio` provides an event loop for asynchronous programming. For example, if we need to make requests without blocking the main thread, we can use the `asyncio` library.

The asyncio module allows for the implementation of asynchronous programming using a combination of the following elements:

- Event loop: The asyncio module allows an event loop per process.
- Coroutines: A coroutine is a generator that follows certain conventions. Its most interesting feature is that it can be suspended during execution to wait for external processing (the some routine in I/O) and return from the point it had stopped when the external processing was done.

- Futures: Futures represent a process that has still not finished. A future is an object that is supposed to have a result in the future and represents uncompleted tasks.
- Tasks: This is a subclass of `asyncio` .Future that encapsulates and manages coroutines. We can use the asyncio.Task object to encapsulate a coroutine.

The most important concept within `asyncio` is the event loop. An event loop allows you to write asynchronous code using either callbacks or coroutines. The keys to understanding `asyncio` are the terms of coroutines and the event loop. **Coroutines** are stateful functions whose execution can be stopped while another I/O operation is being executed. An event loop is used to orchestrate the execution of the coroutines.

To run any coroutine function, we need to get an event loop. We can do this with

```
loop = asyncio.get_event_loop()
```

This gives us a `BaseEventLoop` object. This has a `run_until_complete` method that takes in a coroutine and runs it until completion. Then, the coroutine returns a result. At a low level, an event loop executes the `BaseEventLoop.rununtilcomplete(future)` method.

Share Improve this answer Follow

answered Oct 16, 2021 at 20:27

Yilmaz

40.8k 15 172 225